

Use Cases:

UC1: Customer Registration and Account Management

- **Actors:** Customer
- **Preconditions:**
 - The customer has access to the app or website.
 - A valid email address and phone number are available for verification.
- **Main Flow:**
 1. The customer selects "Create Account."
 2. The system displays a registration form.
 3. The customer enters personal information (name, email, phone).
 4. The customer creates a secure password.
 5. The system sends a verification email or SMS.
 6. The customer verifies their account.
 7. The system activates the account and displays a welcome message.
- **Alternative Flows:**
 - **[E1]** If the email or phone number already exists, the system prompts the user to log in or reset their password.
 - **[E2]** If verification fails, the system offers to resend the verification code.
 - **[E3]** If required fields are missing, the system highlights the incomplete information.

UC2: Browse Menu and View Item Details

- **Actors:** Customer
- **Preconditions:**
 - The customer has access to the app.
- **Main Flow:**
 1. The customer opens the app and navigates to the restaurant list.
 2. The customer selects a restaurant.

3. The system displays the categorized menu with images and prices.
4. The customer selects a specific item.
5. The system displays a detailed view of the item, including ingredients and allergens.

- **Alternative Flows:**

- **[E1]** If an item is temporarily unavailable, the system displays an "out of stock" status.

UC3: Add Items to Cart and Manage Orders

- **Actors:** Customer

- **Preconditions:**

- The customer is logged in.
 - Menu items are available for ordering.

- **Main Flow:**

1. The customer selects a menu item and its quantity.
2. The customer adds the item to their cart.
3. The system updates the cart with the item and running total.
4. The customer can modify quantities or remove items.
5. The customer can add special instructions for preparation.

- **Alternative Flows:**

- **[E1]** If an item becomes unavailable after being added to the cart, the system notifies the customer.

UC4: Process Payment and Complete Order

- **Actors:** Customer

- **Preconditions:**

- The customer has items in their cart.
 - A valid payment method is available.

- **Main Flow:**

1. The customer reviews the order summary and pricing.
2. The customer selects their payment method.
3. The customer applies any discounts or coupons.
4. The customer selects a tip amount for the driver.
5. The customer confirms and processes the payment.
6. The system generates an order confirmation number.
7. The system notifies the restaurant of the new order.

- **Alternative Flows:**

- **[E1]** If the payment fails, the system prompts the customer to retry or use an alternative payment method.
- **[E2]** If the order fails to be placed due to restaurant issues, the system suggests a different restaurant or time.

UC5: Track Order Status and Receive Notifications

- **Actors:** Customer, Restaurant Staff, Delivery Driver

- **Preconditions:**

- A confirmed order has been placed.

- **Main Flow:**

1. The system displays the order status dashboard to the customer.
2. The system sends status updates as the order progresses (e.g., "Received," "Preparing," "Ready").
3. The customer receives a push notification when the order status changes.

- **Alternative Flows:**

- **[E1]** If there is a delay, the system updates the estimated time and notifies the customer.
- **[E2]** If the order is canceled, the system processes a refund and notifies the customer.

UC6: Ordering Food for Delivery to a Different Address

- **Actors:** Customer
- **Preconditions:**
 - The customer is logged in.
 - The new delivery address is within a restaurant's delivery zone.
- **Main Flow:**
 1. The customer begins a new order.
 2. The customer updates their delivery address to the new location.
 3. The system confirms the new address and updates the list of available restaurants.
 4. The customer selects a restaurant and places the order.
 5. The customer confirms the correct delivery address before payment.
- **Alternative Flows:**
 - **[E1]** If the new address is outside a restaurant's delivery zone, the system informs the customer and prompts them to change the address or select a new restaurant.

UC7: Tracking the Delivery Driver's Location

- **Actors:** Customer, Delivery Driver
- **Preconditions:**
 - An order has been placed and a delivery driver has been assigned.
 - The driver's location is being tracked via GPS.
- **Main Flow:**
 1. The customer views their order status screen.
 2. The system displays a map with the driver's real-time location.
 3. The customer can see the estimated time of arrival.
 4. The system sends a notification when the driver is close.
- **Alternative Flows:**

- **[E1]** If the driver's GPS is temporarily unavailable, the system displays a message indicating the location is not available and provides an updated ETA.

UC8: Rating the Restaurant and Delivery Driver

- **Actors:** Customer
- **Preconditions:**
 - The order has been completed.
 - The customer has not yet provided a rating.
- **Main Flow:**
 1. After the order is completed, the system prompts the customer for feedback.
 2. The customer is presented with an option to rate the restaurant and the driver on a scale.
 3. The customer can provide additional comments.
 4. The system saves the ratings and comments.
- **Alternative Flows:**
 - **[E1]** If the customer closes the app before rating, the system sends a push notification or email reminder later.

UC9: Leaving a Tip for the Delivery Driver

- **Actors:** Customer
- **Preconditions:**
 - The customer is at the checkout screen.
 - The platform allows for tipping.
- **Main Flow:**
 1. At checkout, the system displays tipping options (e.g., 15%, 20%, custom).
 2. The customer selects a tip amount.
 3. The tip is added to the order total.
 4. The payment is processed with the tip included.

- **Alternative Flows:**
 - **[E1]** If the customer enters an invalid tip amount (e.g., a negative number), the system prompts them to enter a valid amount.

UC10: Manage Menu Items and Inventory

- **Actors:** Restaurant Staff
- **Preconditions:**
 - The staff member is authenticated with appropriate permissions.
- **Main Flow:**
 1. The staff member accesses the menu management dashboard.
 2. The staff can create new menu items with descriptions and prices.
 3. The staff can update or remove items.
 4. The staff can mark items as available or unavailable.
 5. The system updates the customer-facing menu in real-time.
- **Alternative Flows:**
 - **[E1]** If an ingredient is low in stock, the system suggests marking related items as unavailable.

UC11: View and Process Customer Orders

- **Actors:** Restaurant Staff
- **Preconditions:**
 - Customer orders exist in the system.
- **Main Flow:**
 1. The staff views the incoming orders dashboard.
 2. The staff sees the order details and special instructions.
 3. The staff confirms the order and begins preparation.
 4. The staff updates the order status as it progresses.
 5. The staff marks the order as "Ready for Pickup" when complete.

- **Alternative Flows:**

- **[E1]** If preparation time exceeds the estimate, the staff can update the estimated time to notify the customer.

UC12: Using DoorDash Drive (if applicable) to Earn Money Delivering

- **Actors:** Delivery Driver

- **Preconditions:**

- The individual has been approved as a driver.
- The driver has a vehicle and a smartphone.

- **Main Flow:**

1. The driver logs into the driver app.
2. The driver sets their status to "Available."
3. The system sends an order request with details (restaurant, customer location, payout).
4. The driver accepts the request.
5. The driver navigates to the restaurant, picks up the order, and delivers it to the customer.
6. The driver marks the order as completed in the app.
7. The system processes the payment and updates the driver's earnings.

- **Alternative Flows:**

- **[E1]** If the driver declines an order, the system sends the request to another driver.

UC13: Student Meal Plan Integration

- **Actors:** Customer (Student)

- **Preconditions:**

- The student is enrolled with an active meal plan.
- The meal plan system integration is active.

- **Main Flow:**

1. The student logs in with university credentials.
2. The system displays the student's meal plan balance.
3. The student browses the menu with items marked as "meal plan eligible."
4. The student adds items to the cart and sees a real-time deduction preview.
5. The student confirms the order using meal plan credits.
6. The system automatically deducts the cost from their university account.

- **Alternative Flows:**

- **[E1]** If the meal plan balance is insufficient, the system offers alternative payment options or requires partial payment.
- **[E2]** If the meal plan system is offline, the system queues the transaction for later processing.

UC14: Flexible Student Payment Options

- **Actors:** Customer (Student)

- **Preconditions:**

- The student has multiple payment sources available.
- Each payment method has a sufficient balance for partial payment.

- **Main Flow:**

1. The student adds items to their cart and proceeds to payment.
2. The system displays all available payment options.
3. The student selects a "split payment" option.
4. The student allocates the payment across multiple methods (e.g., 50% meal plan, 50% credit card).
5. The system validates that each source has sufficient funds.
6. The system processes the payments sequentially.

- **Alternative Flows:**

- **[E1]** If one payment method fails, the system notifies the student and offers to re-allocate the remaining balance to another method.

UC15: Delivery Driver Registration

- **Actor:** Prospective Driver
- **Preconditions:**
 - The individual has access to the app or website.
 - The individual has the necessary information for registration, such as a valid driver's license, vehicle details, and personal information.
- **Main Flow:**
 - The prospective driver selects the "Become a Driver" option on the app or website.
 - The system displays the driver registration form.
 - The driver enters their personal details, including name, contact information, and driver's license number.
 - The driver provides vehicle information and other required documentation.
 - The system processes the application and runs a background check.
 - Once approved, the system adds the driver to the database. The system notifies the driver that their account is active and they can begin receiving delivery requests.
- **Alternative Flows:**
 - [E1] If the driver's application is incomplete, the system highlights the missing information and prompts them to complete it.
 - [E2] If the background check or provided information does not meet the requirements, the system notifies the driver of their disqualification.
 - [E3] If the driver's registration is pending, the system provides a status update and estimated time for approval.

How We Decided What Not to Do

To narrow down the use cases for our MVP, we first focused on core functionality. This meant we prioritized the primary features of the application, such as customer registration, ordering, payment processing, and order management. This was the essential foundation that needed to be in place for the service to function at all. We also considered the main user roles—customers, restaurants, and delivery drivers—and prioritized use cases that directly catered to their needs.

A key part of our decision was to evaluate the frequency of use for each feature. We determined that focusing on the most commonly used features would provide the most value for our initial launch. By doing this, we were able to filter out less-frequent tasks and concentrate on the core user journey. This helped us streamline the development process and avoid getting bogged down in complex or niche features.

Ultimately, we chose to eliminate use cases that were either too specific, redundant, or not essential to the core functionality of the MVP. For example, features like "Ordering Food for a Special Occasion" or "Utilizing Specific Restaurant Deals or Promotions" were not included in our final list. We determined that these could be added in later phases once the foundational product was stable.

What Negative Impacts This MVP Could Have for Stakeholders

Our MVP, with its limited scope, may disappoint some of our key stakeholders. Customers may be disappointed by the lack of certain features they might expect from a full-fledged food delivery service. For example, they may miss the ability to schedule an order for a future time or to use promo codes. These features are common in the industry and their absence could affect customer satisfaction and retention.

Restaurants also face potential disappointments with our MVP. The platform in its initial state may lack advanced features for managing their operations. They could be concerned about the lack of advanced inventory management features or the inability to manage complex orders through the system. This might make it more difficult for them to handle their day-to-day business through our platform.

Finally, our delivery drivers may also feel the negative impact of the MVP's limited feature set. They could be disappointed by the lack of advanced features designed to make their jobs easier and more efficient. This includes the absence of tools like in-app navigation or optimized route planning. These features are crucial for a seamless and efficient delivery process, and their absence could lead to frustration.

What Changes We Made to the MVP to Appease Stakeholders

To address some of the stakeholders' concerns and mitigate the negative impacts, we considered making several important adjustments to our MVP. We believe these changes will enhance the overall user experience and make the platform more appealing to all parties.

For our restaurant partners, we considered adding a simple inventory management system. This was a direct response to their concern about the lack of this feature. The system would help them keep track of their stock and prevent customers from ordering items that are out of stock, which in turn improves the customer experience.

To address our customers' feedback, we considered introducing a basic scheduling feature. This would allow customers to schedule orders in advance, a feature they might miss from the initial MVP. This addition would partially address their concerns and add a valuable layer of convenience to the service.

Prompt History

Gemini

<https://g.co/gemini/share/bb01e3f867ba>

<https://g.co/gemini/share/8d520ed6cefb>

<https://g.co/gemini/share/aff06174fd16>

<https://g.co/gemini/share/869ab44bd8cc>

Llama

```
user_prompt = f"""
```

```
You are a {ROLE}.
```

```
prepare use cases from the below use cases which are most important and that  
consolidate all the application.
```

```
Just name the use cases for now.
```

```
As you are software developer yourself.
```

```
Identify which are the most important 15-20 use cases for the stakeholders.
```

Originally Generated Use Cases:

Customer Registration and Account Management

Browse Menu and View Item Details

Add Items to Cart and Manage Orders

Process Payment and Complete Order

Track Order Status and Receive Notifications

Manage Menu Items and Inventory

View and Process Customer Orders

Handle Customer Pickup and Order Completion

Generate Reports and Analytics

Manage Customer Support and Issue Resolution

Student Meal Plan Integration

Flexible Student Payment Options

Additional Use Cases Generated After Context:

Ordering from a Favorite Local Restaurant

Ordering from a Chain Restaurant Not Near the User's Location

Ordering a Specific Dish from a Restaurant Menu

Ordering Multiple Items from the Same Restaurant

Ordering Food for Multiple People

Ordering Food for Delivery to a Different Address

Ordering Food for a Specific Time in the Future

Ordering Food for a Special Occasion

Using a Promo Code to Reduce the Cost of the Order

Ordering Food with Dietary Restrictions

Ordering Food with Specific Allergy Information

Using DoorDash's Search Function to Find a Restaurant

Using DoorDash's Filters to Find Restaurants

Browsing Restaurants by Location

Ordering from a Restaurant for the First Time

Re-ordering a Previous Order

Modifying a Previous Order

Adding Special Instructions to the Order

Contacting Restaurant Support via the App

Checking Order Status

Tracking the Delivery Driver's Location

Rating the Restaurant and Delivery Driver

Leaving a Tip for the Delivery Driver

Using Multiple Payment Methods

Utilizing DoorDash's Customer Service for Order Issues

Creating a DoorDash Account

Updating Account Information

Managing Saved Addresses

Managing Saved Payment Methods

Viewing Order History

Contacting DoorDash Customer Support

Using DoorDash Drive (if applicable) to Earn Money Delivering

Participating in DashPass (subscription) Benefits

Utilizing Specific Restaurant Deals or Promotions

Utilizing the App's Features for Catering Orders

Using the App to Schedule Recurring Orders.

"""

```
user_prompt = f"""
```

You are a {ROLE}.

prepare use cases from the below use cases which are most important and that consolidate all the application.

Just name the use cases for now.

As you are software developer yourself.

You have consolidated all the use cases in one file which is {pdf_text} and also from your understanding, now can you please provide me their in detail

->precondition

->main flow

->sub flow

->alternative flow

please generate for all 20 the use cases

"""

```
user_prompt = f"""
```

You are a {ROLE}.

prepare use cases from the below use cases which are most important and that consolidate all the application.

Just name the use cases for now.

As you are software developer yourself.

You have consolidated all the use cases which are
Originally Generated Use Cases:

Customer Registration and Account Management

Browse Menu and View Item Details

Add Items to Cart and Manage Orders

Process Payment and Complete Order

Track Order Status and Receive Notifications

Manage Menu Items and Inventory

View and Process Customer Orders

Handle Customer Pickup and Order Completion

Generate Reports and Analytics

Manage Customer Support and Issue Resolution

Student Meal Plan Integration

Flexible Student Payment Options

Additional Use Cases Generated After Context:

Ordering from a Favorite Local Restaurant

Ordering from a Chain Restaurant Not Near the User's Location

Ordering a Specific Dish from a Restaurant Menu

Ordering Multiple Items from the Same Restaurant

Ordering Food for Multiple People

Ordering Food for Delivery to a Different Address

Ordering Food for a Specific Time in the Future

Ordering Food for a Special Occasion

Using a Promo Code to Reduce the Cost of the Order

Ordering Food with Dietary Restrictions

Ordering Food with Specific Allergy Information

Using DoorDash's Search Function to Find a Restaurant

Using DoorDash's Filters to Find Restaurants

Browsing Restaurants by Location

Ordering from a Restaurant for the First Time

Re-ordering a Previous Order

Modifying a Previous Order

Adding Special Instructions to the Order

Contacting Restaurant Support via the App

Checking Order Status

Tracking the Delivery Driver's Location

Rating the Restaurant and Delivery Driver

Leaving a Tip for the Delivery Driver

Using Multiple Payment Methods

Utilizing DoorDash's Customer Service for Order Issues

Creating a DoorDash Account

Updating Account Information

Managing Saved Addresses

Managing Saved Payment Methods

Viewing Order History

Contacting DoorDash Customer Support

Using DoorDash Drive (if applicable) to Earn Money Delivering

Participating in DashPass (subscription) Benefits

Utilizing Specific Restaurant Deals or Promotions

Utilizing the App's Features for Catering Orders

Using the App to Schedule Recurring Orders.

in one file which is `{pdf_text}` so provide Reflection document (max 2 pages):

How did you decide what NOT to do?

What negative impacts or disappointments this MVP could have for your stakeholders.

What changes you made (and why) to the MVP to appease at least some of the stakeholders.

"""